Artificial Intelligence

BS (CS) Spring 2026

Lab 08 Manual



Learning Objectives

1. Understand the concept of Constraint Satisfaction Problems (CSP)
2. Identify variables, domains, and constraints
3. Explain examples like map coloring, Sudoku, and N-Queens
4. Apply CSP solving techniques
5. Understand heuristics (MRV, LCV, etc.)
6. Learn consistency techniques (Node & Arc consistency)
7. Understand backtracking search

Introduction to CSP

Constraint Satisfaction Problems (CSPs) are widely used in Artificial Intelligence for solving complex problems by applying constraints. They transform real-world problems into structured formats consisting of variables, domains, and constraints.

Applications of CSP

1. Scheduling
2. Timetabling
3. Map coloring
4. Sudoku
5. Resource allocation
6. N-Queens problem

Definition of CSP

A CSP consists of:

Variables: X={X1,X2,X3,...,Xn}X = \{X_1, X_2, X_3, ..., X_n\}X={X1,X2,X3,...,Xn}

Domains:D={D1,D2,D3,...,Dn}D = \{D_1, D_2, D_3, ..., D_n\}D={D1,D2,D3,...,Dn}

Constraints: A set of rules restricting possible values

Goal: Assign values to variables such that all constraints are satisfied.

Key Concepts

- **Consistent Assignment:** Does not violate constraints
- **Complete Assignment:** All variables assigned
- **Solution:** Complete + consistent assignment

Examples of CSPs

Map Coloring

Assign colors to regions
Adjacent regions must have different colors

Example:

Variables: A, B, C, D
Domain: {Red, Green, Blue}
Constraint: Adjacent regions ≠ same color

Sudoku

Each row → unique numbers
Each column → unique numbers

Each sub-grid → unique numbers

N-Queens Problem

Place N queens on an N×N board
No two queens attack each other

Understanding CSP Components

Experiment 1:

Problem

Model a scheduling problem using CSP.

Steps

1. Define variables (tasks)
2. Define domains (time slots)
3. Define constraints (no overlap)

Pseudocode: CSP Representation

Define Variables V = {T1, T2, T3}

Define Domains:

D(T1) = {9AM, 10AM}
D(T2) = {9AM, 11AM}
D(T3) = {10AM, 11AM}

Define Constraints:

T1 ≠ T2
T2 ≠ T3

Return CSP(V, D, Constraints)

Experiment 2: Binary CSP (Map Coloring)

Problem

Color regions so adjacent regions have different colors.

Pseudocode: Map Coloring

Define Regions = {A, B, C}

Define Colors = {Red, Green, Blue}

For each region:
  Assign a color

Constraints:
  A ≠ B
  B ≠ C
  A ≠ C

Check all assignments:
  If constraints satisfied → Solution found

Experiment 3: Backtracking Algorithm

Objective

Solve CSP using Backtracking

Steps

1. Select variable
2. Assign value
3. Check constraints
4. If failure → backtrack

Pseudocode: Backtracking

```
function BACKTRACK(assignment):
   if assignment is complete:
      return assignment

   variable = select_unassigned_variable()

   for value in domain(variable):
      if value is consistent:
         assign value

         result = BACKTRACK(assignment)

         if result ≠ failure:
            return result

         remove assignment (backtrack)

   return failure
```

Experiment 4: Forward Checking

Objective

Improve backtracking by reducing domain early

Pseudocode: Forward Checking

```
function FORWARD_CHECK(variable, value):
   assign variable = value

   for each neighbor of variable:
      remove inconsistent values from neighbor domain

      if domain becomes empty:
         return failure

   return success
```

Experiment 5: Constraint Propagation (AC-3 Algorithm)

Objective

Enforce arc consistency

Pseudocode: AC-3

```
function AC3(queue):
   while queue is not empty:
      (Xi, Xj) = remove edge from queue

      if REVISE(Xi, Xj):
         if domain(Xi) is empty:
            return failure

         for each neighbor Xk of Xi:
            add (Xk, Xi) to queue

   return success


function REVISE(Xi, Xj):
   revised = false

   for each value x in domain(Xi):
      if no value y in domain(Xj) satisfies constraint:
         remove x from domain(Xi)
         revised = true

   return revised
```

Experiment 6: Solving Sudoku using CSP

Problem Description

- Variables → Cells
- Domain → {1–9}
- Constraints → No repetition in row, column, subgrid

Pseudocode: Sudoku Solver

```
function SOLVE_SUDOKU(grid):
    if all cells filled:
        return true

    (row, col) = find_empty_cell()

    for num in 1 to 9:
        if is_valid(row, col, num):
            grid[row][col] = num

            if SOLVE_SUDOKU(grid):
                return true

            grid[row][col] = 0   // backtrack

    return false


function is_valid(row, col, num):
    check row constraint
    check column constraint
    check subgrid constraint

    return true if valid else false
```

Experiment 7: Dynamic CSP

Concept

Variables or constraints change during execution.

Pseudocode: Dynamic CSP Handling

```
while system is running:
    if constraint changes:
        update CSP model

    re-evaluate assignments

    if conflict:
        re-solve CSP
```

Components of a CSP

Variables

Unknown elements that need values

Domains

Possible values for each variable

Constraints

Rules limiting value combinations

Example

Let:

- X={A,B}X = \{A, B\}X={A,B}
- D(A)=D(B)={1,2,3}D(A) = D(B) = \{1,2,3\}D(A)=D(B)={1,2,3}
- Constraint: A≠BA \ne BA=B

✓ Valid: (A=1, B=2)
✗ Invalid: (A=2, B=2)

Techniques to Solve CSPs

Backtracking Algorithm

Assign values one by one
Backtrack if constraint fails

Heuristics

Used to improve efficiency:

MRV (Minimum Remaining Values):
Choose variable with fewest legal values

Most Constraining Variable:
Break ties by selecting variable affecting most others

LCV (Least Constraining Value):
Choose value that leaves most options

Consistency Techniques

Node Consistency:
Ensures unary constraints are satisfied

Arc Consistency (AC-3):
Ensures valid values exist between connected variables

Pseudocodes

MRV (Minimum Remaining Values)

```
function SelectVariableMRV(UnassignedVariables):
  bestVariable ← null
  minDomain ← infinity

  for each variable in UnassignedVariables:
    if DomainSize(variable) < minDomain:
      minDomain ← DomainSize(variable)
      bestVariable ← variable

  return bestVariable
```

Most Constraining Variable

```
function SelectMostConstraining(Candidates):
  bestVariable ← null
  maxConstraints ← -1

  for each variable in Candidates:
    count ← NumberOfConstraintsOnNeighbors(variable)
    if count > maxConstraints:
      maxConstraints ← count
```

```
        bestVariable ← variable


  return bestVariable
```

Least Constraining Value (LCV)

```
function OrderValuesLCV(variable):
  for each value in Domain(variable):
    score(value) ← NumberOfValuesRuledOut(value)


  return values sorted in ascending order of score
```

Node Consistency

```
function NodeConsistency(Variables):
  for each variable in Variables:
    for each value in Domain(variable):
      if value violates unary constraint:
        remove value
```

Arc Consistency (AC-3)

```
function AC3(CSP):
  queue ← all arcs


  while queue not empty:
    (Xi, Xj) ← remove first


    if Revise(Xi, Xj):
```

```
        if Domain(Xi) is empty:
            return false

        for each neighbor Xk ≠ Xj:
            add (Xk, Xi) to queue

    return true
```

Backtracking Search

```
function Backtrack(assignment, CSP):
    if assignment is complete:
        return assignment

    variable ← select unassigned variable

    for each value in domain:
        if consistent:
            assign value
            result ← Backtrack()

            if result ≠ failure:
                return result

            remove assignment

    return failure
```